PROJECT REPORT

Containerized RTL Verification and PPA Analysis Framework for Digital VLSI Designs

A Docker-based workflow for RTL linting, functional simulation, synthesis, timing and power estimation, and design comparison.

Project detail	Description
Domain	Digital VLSI design automation
Demonstration block	32-bit Arithmetic Logic Unit (ALU)
Technology library	SkyWater SKY130 HD, tt 25°C, 1.8 V
Tools	Docker, Verilator, Icarus Verilog, Yosys, OpenSTA
Report date	9 October 2026

Student name, institution, guide, and submission details can be added to match the college report template.

Abstract

This project implements a repeatable, containerized workflow for evaluating digital RTL designs. A 32-bit ALU is used as the demonstration design. The framework runs linting and functional simulation, synthesizes RTL to a gate-level netlist using Yosys and the Sky130 HD standard-cell library, obtains pre-layout timing and estimated power results, and compares a baseline implementation with an optimized implementation. Docker provides a consistent tool environment, while shell scripts automate the stages and save logs and reports.

Both versions passed eight directed functional test cases in the recorded runs. The optimized version produced 464 mapped cells compared with 527 for the baseline, 7.80% lower reported cell area, 30.17% lower reported critical-path delay, and 1.96% lower estimated power. These are current synthesis/pre-layout estimates, not post-layout or silicon measurements.

1. Introduction and Problem Statement

RTL evaluation typically involves multiple tools and stages. Manual execution can lead to inconsistent environments, repeated commands, and difficult-to-compare results. A repeatable workflow should execute checks in a consistent order and preserve outputs that support design decisions.

Problem statement

Develop a Docker-based automation framework that executes RTL linting, functional simulation, logic synthesis, timing and power estimation, and PPA comparison for digital designs, while saving generated logs, netlists, and reports in a consistent project structure.

Proposed solution

The project uses a Docker image and shell scripts to run the same general evaluation flow on baseline and optimized ALU RTL. It collects results and reports changes in mapped cell count, area, critical-path delay, slack, and estimated power.

2. Objectives and Scope

• Automate linting and functional simulation.

• Run tools in a consistent Docker environment.

• Synthesize RTL using Yosys and the Sky130 HD standard-cell library.

• Collect pre-layout timing and estimated power metrics.

• Compare baseline and optimized implementations.

• Save logs, netlists, statistics, waveforms, and comparison reports.

Scope boundary

The demonstrated project covers RTL-level verification and synthesis-based PPA analysis. The evidence does not establish detailed placement, clock-tree synthesis, routing, extracted parasitics, post-layout signoff, or fabricated-silicon measurement.

3. Tools and System Architecture

The workflow connects RTL source files to verification, synthesis, timing/power estimation, and report generation. Docker helps make the tool environment repeatable; scripts provide orchestration.

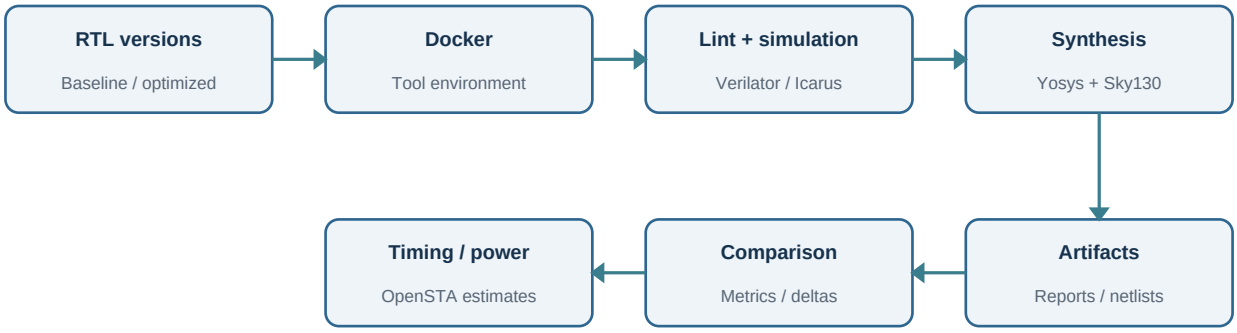

Figure 1. Conceptual architecture of the automated evaluation flow.

Tool / component	Role
Docker	Runs EDA tools in a consistent container image.
Verilator	Checks RTL for lint warnings and errors.
Icarus Verilog	Compiles and runs the ALU testbench.
VCD waveform	Records signal transitions for waveform inspection.
Yosys	Synthesizes RTL and maps logic to Sky130 HD cells.
OpenSTA	Reports timing metrics under configured constraints.
Shell scripts / Makefile	Automate stages and compare the two RTL variants.
Reports / results folders	Store logs, netlists, statistics, and comparison outputs.

Key project files

`rtl/alu.v`, `rtl/alu_optimized.v`, `tb/alu_tb.v`, `scripts/simulate.sh`, `scripts/run_comparison.sh`, and `reports/ppa/comparison_report.txt`. Generated results are stored under `results/baseline/` and `results/optimized/`.

4. Demonstration Design: 32-bit ALU

The ALU supports eight operations. The current directed testbench exercises each operation with example operands A = 10 and B = 5; shift operations use a shift amount of one.

Operation	Expected example result
ADD	10 + 5 = 15
SUB	10 – 5 = 5
AND	0x0000000A AND 0x00000005 = 0x00000000
OR	0x0000000A OR 0x00000005 = 0x0000000F
XOR	0x0000000A XOR 0x00000005 = 0x0000000F
NOT A	~0x0000000A = 0xFFFFFFF5
Left shift	0x0000000A << 1 = 0x00000014

Operation	Expected example result
Right shift	0x0000000A >> 1 = 0x00000005

Optimization idea

The optimized RTL shares the arithmetic datapath for addition and subtraction. For subtraction, the second operand is conditionally inverted and a carry-in of one is applied, implementing two's-complement subtraction. Hardware sharing can reduce duplicated arithmetic logic, but the final outcome depends on RTL, synthesis settings, target library, and constraints.

5. Methodology and Execution Flow

1. Lint. Run Verilator on each RTL variant and record warning/error counts.

2. Compile and simulate. Use Icarus Verilog to compile the selected RTL with the testbench and run it.

3. Check simulation summary. The updated simulation script checks for a test summary, zero reported failures, a completion marker, and compile/simulator errors.

4. Synthesize. Use Yosys with the Sky130 HD liberty file to generate a mapped netlist and statistics.

5. Analyze timing and power. Run the configured timing and power-estimation scripts.

6. Save results. Store baseline and optimized outputs in separate result folders.

7. Compare. Calculate changes and write reports/ppa/comparison_report.txt.

Top-level command

```
cd ~/docker-vlsi-framework
make all
```

The successful recorded run ended with COMPARISON WORKFLOW COMPLETE. The simulation script was separately tested with a temporary testbench reporting one failed test; it correctly returned SIMULATION STATUS: FAIL and exit code 1. The temporary testbench was then removed, and the normal comparison workflow passed again.

6. Verification Results

Verification item	Baseline	Optimized	Interpretation
Lint warnings	0	0	No warnings in captured runs.
Lint errors	0	0	No errors in captured runs.
Tests passed	8	8	All eight directed cases passed.
Tests failed	0	0	No failures reported.
Simulation status	PASS	PASS	Simulation completed normally.

Failure-detection check

A temporary testbench printed TOTAL TESTS = 1, PASSED = 0, FAILED = 1. The updated simulation script correctly printed a failure status and exited with code 1. This verifies the script's reported-failure path, not detection of an actual arithmetic defect in the ALU.

Coverage caveat

The testbench covers one directed example per operation. This is useful smoke-test coverage but is not exhaustive. Additional boundary values, randomized operands, assertions, and coverage metrics would strengthen verification.

7. Synthesis and PPA Comparison

Metric	Baseline	Optimized	Change
Mapped cell count	527	464	11.95% reduction
Cell area	3080.4544 µm²	2840.2240 µm²	7.80% reduction
Critical path delay	10.11 ns	7.06 ns	30.17% reduction
Worst slack	1.89 ns	4.94 ns	3.05 ns higher
Estimated power	0.102 mW	0.100 mW	1.96% reduction

Interpretation

The recorded outputs show fewer mapped cells and lower reported area for the optimized RTL. The critical path is shorter and worst slack is higher under the current timing setup. Estimated power changes only slightly. These comparisons are meaningful when the same library, constraints, tool configuration, and assumptions are applied to both versions.

• Mapped cell area is not the same as die/core area of a completed chip.

• Timing is pre-layout and uses the current constraints and ideal-clock assumptions.

• Power is estimated using configured default activity; it is not a post-layout or silicon measurement.

• Netlist file size on disk is not the same metric as mapped cell area.

8. Conclusion, Limitations, and Future Work

Conclusion

The project demonstrates a containerized, script-driven workflow for linting, functional simulation, synthesis, timing/power estimation, and comparison of two digital RTL implementations. Both ALU versions passed the current eight directed functional tests. The optimized version showed 11.95% fewer mapped cells, 7.80% lower reported area, 30.17% lower critical-path delay, and 1.96% lower estimated power in the recorded run.

Limitations

• Functional coverage is limited to eight directed cases and a small set of operands.

• The failure-detection test was run directly through simulate.sh; failure propagation through the entire make all flow has not been deliberately demonstrated.

• Timing and power are pre-layout estimates; physical-design and post-layout signoff are outside the demonstrated flow.

• Results depend on tool versions, library files, constraints, and script settings.

Future work

• Add randomized and boundary-value tests and stronger self-checking assertions.

• Add regression tests for compile errors, simulator errors, and functional failures.

• Verify that make all stops and returns a non-zero status if any stage fails.

• Record tool versions and configuration metadata in reports.

• Archive separate baseline and optimized waveforms.

• Extend to physical design with placement, routing, extraction, and post-layout analysis if required.

9. Demonstration Walkthrough

Step	Evidence to show	Explanation
1	Project folders	Show rtl/, tb/, scripts/, lib/, reports/, results/, waveforms/.
2	Run make all	Show both variants and final completion line.
3	Simulation output	Show eight PASS lines, 8 passed, 0 failed.
4	Synthesis statistics	Open both alu_stat.txt files and mapped netlists.
5	Gate-level schematic	Open results/optimized/synthesis/alu_schematic.svg in a browser/viewer.
6	PPA comparison	Show reports/ppa/comparison_report.txt and explain improvements.
7	Failure detection	Show terminal evidence that the temporary failure test returned exit code 1.

Suggested viva / demo summary

> “This project automates the RTL evaluation flow using Docker. It checks RTL with linting, verifies behavior with simulation, synthesizes the design to Sky130 standard cells, estimates timing and power, and compares baseline and optimized implementations. Both versions passed eight directed tests. The optimized version reduced mapped area and critical-path delay in the current pre-layout flow. The simulation script also detects a reported test failure.”

Appendix A. Evidence Checklist

Actual screenshots were not supplied for this report, so the following checklist identifies evidence to capture from the project environment and insert into the final submission as required by your college format.

ID	Evidence	Screenshot should show
A1	Project structure	Project directories in terminal or File Explorer.
A2	Successful run	make all output ending with COMPARISON WORKFLOW COMPLETE.
A3	Baseline simulation	Eight PASS lines and 0 failed.
A4	Optimized simulation	Eight PASS lines and 0 failed.
A5	Failure detection	FAIL status and exit code 1 from temporary test.
A6	Synthesis stats	Cell count and area for each version.
A7	Schematic	Optimized alu_schematic.svg open in viewer.
A8	PPA report	Baseline-to-optimized comparison report.
A9	Waveform	alu.vcd open in GTKWave or another VCD viewer, if available.

Useful artifact paths

Comparison report: `reports/ppa/comparison_report.txt`
Optimized schematic: `results/optimized/synthesis/alu_schematic.svg`
Simulation waveform: `waveforms/alu.vcd`
Top-level command: `make all`

End of report